C# اليوم 5 · الأسبوع 1 · ساعتان · دورة أساسيات البرمجة

العمليات والشروط + اختبار منتصف الدورة

أهداف التعلم

في نهاية الدرس يستطيع المتدرب أن:

- يستخدم العمليات الحسابية + − * / % بشكل صحيح.
- يقارن القيم بـ == و!= و> و<.
- يجمع الشروط بـ && و|| و!.
- يكتب قرارات بـ if / else if / else.
- يستخدم switch لاختيار من عدة حالات.

توزيع وقت الحصة

0:00 – 0:10	مراجعة المتغيرات والإدخال
0:10 – 0:45	شرح العمليات الحسابية والمنطقية وif وswitch
0:45 – 1:20	تمارين عملية على الشروط
1:20 – 1:30	استراحة
1:30 – 1:55	اختبار منتصف الدورة (HTML + أساسيات C#)
1:55 – 2:00	ملاحظات ختام الأسبوع الأول

شرح الدرس

1. العمليات الحسابية

`%` يعطي **باقي القسمة**، ومفيد لمعرفة الزوجي والفردي. انتبه: قسمة عددين صحيحين تعطي عددًا صحيحًا: `7 / 2` = 3.

C#

```
int a = 17, b = 5;
Console.WriteLine(a + b);   // 22
Console.WriteLine(a - b);   // 12
Console.WriteLine(a * b);   // 85
Console.WriteLine(a / b);   // 3
Console.WriteLine(a % b);   // 2
Console.WriteLine(7.0 / 2); // 3.5
```

ماذا يفعل هذا الكود؟ يجرب البرنامج العمليات الحسابية الأساسية على العددين 17 و 5 ويطبع ناتج كل عملية في سطر. يرى الطالب: 22 ثم 12 ثم 85 ثم 3 ثم 2 ثم 3.5.

`int a = 17, b = 5;` نُنشئ متغيرين من نوع int (عدد صحيح) في سطر واحد، ونفصل بينهما بفاصلة: a قيمته 17 و b قيمته 5.

`Console.WriteLine(a + b);  // 22`	العلامة + تجمع العددين، و WriteLine يطبع الناتج 22. النص بعد // تعليق يوضح النتيجة المتوقعة ولا يُنفذ.
`Console.WriteLine(a - b);  // 12`	العلامة - للطرح، فيُطبع 12.
`Console.WriteLine(a * b);  // 85`	العلامة * للضرب في C# (وليس ×)، فيُطبع 85.
`Console.WriteLine(a / b);  // 3`	العلامة / للقسمة. لكن عندما يكون العددان صحيحين int تكون القسمة «صحيحة»: يُحذف الكسر وتبقى النتيجة 3 بدلًا من 3.4.
`Console.WriteLine(a % b);  // 2`	العلامة % تعطي «باقي القسمة»: 17 مقسومة على 5 تساوي 3 والباقي 2، فيُطبع 2. هذه العملية مفيدة جدًا، مثلًا لمعرفة هل العدد زوجي أم فردي.
`Console.WriteLine(7.0 / 2); // 3.5`	كتبنا 7.0 بفاصلة عشرية فأصبح عددًا عشريًا double، وعندما يكون أحد طرفي القسمة عشريًا تحتفظ النتيجة بالكسر فيُطبع 3.5.

2. المقارنة والمنطق

العملية	المعنى
`==`	يساوي
`!=`	لا يساوي
`> < >= <=`	أكبر، أصغر، أكبر أو يساوي، أصغر أو يساوي
`&&`	و (الشرطان صحيحان)
`||`	أو (أحدهما صحيح)
`!`	نفي

3. الجملة الشرطية if

ينفذ البرنامج الجزء الأول الذي يتحقق شرطه فقط، ويتجاهل الباقي.

```
C#

Console.Write("أدخل الدرجة: ");
int score = int.Parse(Console.ReadLine());

if (score >= 90)
    Console.WriteLine("ممتاز");
else if (score >= 80)
    Console.WriteLine("جيد جدًا");
```

```
else if (score ⩾ 60)
    Console.WriteLine("ناجح");
else
    Console.WriteLine("راسب");
```

```
Output ▸ 85 :أدخل الدرجة
جيد جدًا
```

ماذا يفعل هذا الكود؟ يطلب البرنامج درجة الطالب ثم يطبع تقديرًا واحدًا حسبها: ممتاز أو جيد جدًا أو ناجح أو راسب. مثلًا الدرجة 85 تطبع «جيد جدًا».

`Console.Write("أدخل الدرجة: ");`	نطبع طلبًا للمستخدم ليكتب الدرجة، ويبقى المؤشر في نفس السطر.
`int score = int.Parse(Console.ReadLine());`	ReadLine يعيد ما كتبه المستخدم كنص، و int.Parse يحوّله إلى عدد صحيح حتى نستطيع مقارنته بأعداد أخرى. نحفظه في score.
`if (score ⩾ 90)`	الكلمة if تعني «إذا»: تفحص الشرط بين القوسين، فإن كان صحيحًا تُنفذ التعليمة التي تحته. العلامة >= تعني «أكبر من أو يساوي».
`Console.WriteLine("ممتاز");`	هذا السطر يُنفذ فقط إذا كانت الدرجة 90 أو أكثر. عندما لا نستخدم الأقواس { } يتبع if سطر واحد فقط هو الذي بعده مباشرة.
`else if (score ⩾ 80)`	else if تعني «وإلا، إذا...»: لا يُفحص هذا الشرط إلا إذا كان الشرط السابق خاطئًا. أي أننا نصل هنا فقط إذا كانت الدرجة أقل من 90.
`Console.WriteLine("جيد جدًا");`	يُطبع «جيد جدًا» إذا كانت الدرجة من 80 إلى 89.
`else if (score ⩾ 60)`	شرط ثالث يُفحص فقط إذا فشل الشرطان السابقان، أي الدرجة أقل من 80.
`Console.WriteLine("ناجح");`	يُطبع «ناجح» إذا كانت الدرجة من 60 إلى 79.
`else`	else وحدها تعني «في كل الحالات الأخرى» ولا تحتاج شرطًا. تُنفذ فقط إذا كانت كل الشروط السابقة خاطئة، أي الدرجة أقل من 60.
`Console.WriteLine("راسب");`	يُطبع «راسب». لاحظ أن البرنامج ينفذ فرعًا واحدًا فقط من السلسلة: أول شرط صحيح ثم يتجاهل الباقي.

خطأ شائع: كتابة = (إسناد) بدل == (مقارنة) داخل الشرط.

4. الجملة switch

تناسب الاختيار من قيم محددة، مثل قوائم البرامج.

```
C#
```

```
Console.Write("(خروج 0، عرض 2، إضافة 1) اختر: ");
string choice = Console.ReadLine();

switch (choice)
{
    case "1":
        Console.WriteLine("إضافة طالب.  ");
        break;
    case "2":
        Console.WriteLine("عرض الطلاب.  ");
        break;
    case "0":
        Console.WriteLine("وداعًا");
        break;
    default:
        Console.WriteLine("اختيار غير صحيح");
        break;
}
```

ماذا يفعل هذا الكود؟ يعرض البرنامج قائمة اختيارات ويقرأ اختيار المستخدم، ثم يطبع رسالة مناسبة: إضافة أو عرض أو وداع، أو «اختيار غير صحيح» إذا كتب شيئًا آخر.

`Console.Write("(خروج 0: ");اختر (1 إضافة، 2 عرض،`	نطبع قائمة الاختيارات المتاحة ليعرف المستخدم ماذا يكتب، ويبقى المؤشر في نفس السطر.
`string choice = Console.ReadLine();`	نقرأ اختيار المستخدم كنص ونحفظه في choice. لم نحوّله إلى عدد لأننا سنقارنه بنصوص مثل "1".
`switch (choice)`	الكلمة switch تعني «بدّل حسب»: تأخذ قيمة choice وتقارنها بعدة قيم محتملة، وتنفذ الجزء المطابق. هي بديل أوضح لسلسلة طويلة من if و else if.
`{`	الأقواس المعقوفة { } تجمع مجموعة من التعليمات في «كتلة» واحدة، وهنا تحدد بداية ونهاية جسم الـ switch.
`case "1":`	case تعني «حالة»: إذا كانت قيمة choice تساوي النص "1" يبدأ التنفيذ من هنا. لاحظ علامة النقطتين : في نهاية السطر.
`Console.WriteLine("إضافة طالب.  ");`	هذه التعليمة تُنفذ عندما يختار المستخدم 1.
`break;`	break تعني «اخرج من الـ switch الآن». في C# يجب أن ينتهي كل case بـ break (أو ما يشبهها) حتى لا ينتقل التنفيذ إلى الحالة التالية، وإلا يرفض المترجم البرنامج.
`case "2":`	الحالة الثانية: إذا كتب المستخدم "2" تُنفذ التعليمات التي تحتها.

`Console.WriteLine("عرض الطلاب. ");`	يُطبع «عرض الطلاب...» ثم تأتي break للخروج.
`case "0":`	الحالة الثالثة: إذا كتب المستخدم "0" للخروج.
`Console.WriteLine("وداعًا");`	يُطبع «وداعًا» ثم break تُنهي الـ switch.
`default:`	default تعني «الحالة الافتراضية»: تُنفذ إذا لم تطابق القيمة أي case من الحالات السابقة، مثل أن يكتب المستخدم 5 أو حرفًا.
`Console.WriteLine("اختيار غير صحيح");`	يُطبع تنبيه للمستخدم بأن ما كتبه ليس من الاختيارات المتاحة، ثم break الأخيرة تُنهي الـ switch.

التمارين العملية

تمرين 1: زوجي أم فردي

سهل

اقرأ عددًا واطبع هل هو زوجي أم فردي.

الحل المقترح:

C#

```
Console.Write("أدخل عددًا: ");
int n = int.Parse(Console.ReadLine());
if (n % 2 == 0)
    Console.WriteLine("زوجي");
else
    Console.WriteLine("فردي");
```

ماذا يفعل هذا الكود؟ يطلب البرنامج عددًا صحيحًا ثم يطبع «زوجي» إذا كان يقبل القسمة على 2، أو «فردي» إذا لم يكن كذلك.

`Console.Write("أدخل عددًا: ");`	نطبع طلبًا للمستخدم ليكتب عددًا.
`int n = int.Parse(Console.ReadLine());`	نقرأ ما كتبه كنص بـ ReadLine، ثم نحوّله إلى عدد صحيح بـ int.Parse ونحفظه في n.
`if (n % 2 == 0)`	n % 2 يحسب باقي قسمة العدد على 2، والعلامة == تعني «هل يساوي؟» للمقارنة، وهي تختلف عن = التي تضع قيمة في متغير. فإذا كان الباقي صفرًا فالعدد زوجي.
`Console.WriteLine("زوجي");`	يُطبع «زوجي» فقط عندما يكون الشرط صحيحًا.

`else`	else تعني «وإلا»: تُنفذ عندما يكون شرط if خاطئًا، أي عندما يكون الباقي 1 (أو -1 للأعداد السالبة).
`Console.WriteLine("فردي");`	يُطبع «فردي» في هذه الحالة.

تمرين 2: تقدير الطالب

متوسط

اقرأ درجة من 100. إن كانت أقل من 0 أو أكبر من 100 اطبع «درجة غير صالحة»، وإلا اطبع التقدير: ممتاز (90+)، جيد جدًا (80+)، جيد (70+)، مقبول (60+)، راسب.

الحل المقترح:

C#

```
Console.Write("الدرجة: ");
int s = int.Parse(Console.ReadLine());

if (s < 0 || s > 100) Console.WriteLine("درجة غير صالحة");
else if (s ⩾ 90) Console.WriteLine("ممتاز");
else if (s ⩾ 80) Console.WriteLine("جيد جدًا");
else if (s ⩾ 70) Console.WriteLine("جيد");
else if (s ⩾ 60) Console.WriteLine("مقبول");
else Console.WriteLine("راسب");
```

ماذا يفعل هذا الكود؟ يطلب البرنامج درجة من 0 إلى 100 ويطبع تقديرها (ممتاز، جيد جدًا، جيد، مقبول، راسب)، ويطبع «درجة غير صالحة» إذا كانت الدرجة خارج هذا المدى.

`Console.Write("الدرجة: ");`	نطبع طلبًا للمستخدم ليكتب الدرجة.
`int s = int.Parse(Console.ReadLine());`	نقرأ النص الذي كتبه ونحوّله إلى عدد صحيح بـ int.Parse، ونحفظه في المتغير s.
`if (s < 0 || s > 100) Console.WriteLine("درجة غير صالحة");`	العلامة || تعني «أو»: يكون الشرط صحيحًا إذا تحقق أحد الجزأين على الأقل، أي الدرجة أقل من 0 أو أكبر من 100. نفحص هذا أولًا لنرفض الدرجات غير المنطقية، ويمكن كتابة التعليمة في نفس سطر if.
`else if (s ⩾ 90) Console.WriteLine("ممتاز");`	نصل إلى هنا فقط إذا كانت الدرجة صالحة. إذا كانت 90 أو أكثر يُطبع «ممتاز».
`else if (s ⩾ 80) Console.WriteLine("جيد جدًا");`	إذا فشلت الشروط السابقة وكانت الدرجة 80 أو أكثر (أي 80 إلى 89) يُطبع «جيد جدًا».
`else if (s ⩾ 70) Console.WriteLine("جيد");`	الدرجات من 70 إلى 79 تعطي «جيد». ترتيب الشروط من الأعلى إلى الأدنى مهم لأن أول شرط صحيح هو الذي يُنفذ.

`else if (s ⩾ 60) Console.WriteLine("مقبول");`	الدرجات من 60 إلى 69 تعطي «مقبول».
`else Console.WriteLine("راسب");`	else الأخيرة تلتقط كل ما تبقى، أي الدرجات من 0 إلى 59، فيُطبع «راسب».

تمرين 3: تحدٍّ: آلة حاسبة

تحدٍّ

اقرأ عددين وعملية (+ أو - أو * أو /) واستخدم switch لطباعة الناتج. لا تسمح بالقسمة على صفر.

الحل المقترح:

C#

```
Console.Write("العدد الأول: ");
double x = double.Parse(Console.ReadLine());
Console.Write("العملية (+ - * /): ");
string op = Console.ReadLine();
Console.Write("العدد الثاني: ");
double y = double.Parse(Console.ReadLine());

switch (op)
{
    case "+": Console.WriteLine(x + y); break;
    case "-": Console.WriteLine(x - y); break;
    case "*": Console.WriteLine(x * y); break;
    case "/":
        if (y == 0) Console.WriteLine("لا يمكن القسمة على صفر");
        else Console.WriteLine(x / y);
        break;
    default: Console.WriteLine("عملية غير معروفة"); break;
}
```

ماذا يفعل هذا الكود؟ آلة حاسبة بسيطة: يطلب البرنامج عددين وعملية حسابية (+ أو - أو * أو /) ثم يطبع النتيجة، ويرفض القسمة على صفر أو أي عملية غير معروفة.

`Console.Write("العدد الأول: ");`	نطبع طلبًا للمستخدم ليكتب العدد الأول.
`double x = double.Parse(Console.ReadLine());`	نقرأ النص ونحوّله إلى عدد عشري بـ double.Parse حتى نقبل أعدادًا بكسور، ونحفظه في x.
`Console.Write("العملية (+ - * /): ");`	نطلب من المستخدم أن يكتب رمز العملية التي يريدها.

`string op = Console.ReadLine();`	نحفظ رمز العملية كنص في المتغير op، ولا نحوّله لأنه رمز وليس عددًا.
`Console.Write("العدد الثاني: ");`	نطلب العدد الثاني.
`double y = double.Parse(Console.ReadLine());`	نقرأ العدد الثاني ونحوّله إلى double ونحفظه في y.
`switch (op)`	switch يقارن قيمة op بكل حالة case، وينفذ الحالة التي تطابق رمز العملية.
`{`	الأقواس المعقوفة { } تجمع التعليمات في كتلة واحدة، وهنا تحدد بداية ونهاية جسم الـ switch.
`case "+": Console.WriteLine(x + y); break;`	إذا كان op هو "+" نطبع ناتج الجمع ثم break للخروج من الـ switch. يمكن كتابة case وتعليماتها و break في سطر واحد.
`case "-": Console.WriteLine(x - y); break;`	إذا كان op هو "-" نطبع ناتج الطرح ثم نخرج.
`case "*": Console.WriteLine(x * y); break;`	إذا كان op هو "*" نطبع ناتج الضرب ثم نخرج.
`case "/":`	حالة القسمة تحتاج عدة أسطر لأننا نريد فحص شيء قبل القسمة.
`if (y == 0) Console.WriteLine("لا يمكن القسمة على صفر");`	نستخدم == للمقارنة: إذا كان العدد الثاني صفرًا نطبع رسالة تحذير. مع النوع double لا يتوقف البرنامج عند القسمة على صفر بل يعطي قيمة غريبة مثل ∞ (Infinity)، لذلك نمنعها بأنفسنا.
`else Console.WriteLine(x / y);`	إذا لم يكن y صفرًا نقسم ونطبع النتيجة بأمان.
`break;`	break تُنهي حالة القسمة وتخرج من الـ switch، وهي ضرورية في C# في نهاية كل case.
`default: Console.WriteLine("عملية غير معروفة"); break;`	default تُنفذ إذا كتب المستخدم رمزًا لا يطابق أي حالة، مثل % أو حرف، فنخبره أن العملية غير معروفة.

اختبر نفسك (3 أسئلة)

1. ما ناتج 10 % 3 ؟

3 | 1 | 0

2. لكي يتحقق الشرط يجب أن يكون الشرطان صحيحين، نستخدم:

|| && !

3. ما الكلمة التي تنهي كل حالة case؟

stop end break

الإجابات: 1) 1 · 2) && · 3) break

الواجب: راجع دروس الأسبوع الأول، واختر فكرة مشروعك النهائي من قائمة المشروع.